

LibQueue: Sistema de Gestão de Biblioteca Acadêmica

Ana Clara M. Viana, Emanuel F. Nogueira, Cláudio Rodolfo S. de Oliveira

Departamento de Ensino Superior (DESUP) - Instituto Federal da Bahia (IFBA)
CEP 45078-900 – Vitória da Conquista/BA – Brasil

{202511190007, 202511190004, claudiorodolfo}@ifba.edu.br

Abstract. *This article presents the development of LibQueue, a library management system focused on controlling books, users, loans, returns, and reservations. The system was developed in Java using JavaFX and the MVC architecture, with data structures applied to information management. The use of a priority queue for reservations stands out, giving preference to teachers over students. The system also allows the management of the collection, loans, returns, and overdue items. The results demonstrate that the solution can contribute to more organized and efficient management of school and academic library activities.*

Resumo. *Este artigo apresenta o desenvolvimento do LibQueue, um sistema de gestão de biblioteca voltado ao controle de livros, usuários, empréstimos, devoluções e reservas. O sistema foi desenvolvido em Java, utilizando JavaFX e a arquitetura MVC, com estruturas de dados aplicadas ao gerenciamento das informações. Destaca-se a utilização de uma fila de prioridade nas reservas, dando preferência aos professores em relação aos alunos. O sistema também permite o controle do acervo, empréstimos, devoluções e atrasos. Os resultados demonstram que a solução pode contribuir para uma gestão mais organizada e eficiente das atividades de bibliotecas escolares e acadêmicas.*

1.Introdução

A informatização de bibliotecas escolares tem sido adotada como alternativa para aprimorar a organização do acervo e a qualidade dos serviços oferecidos aos usuários. Segundo Barbosa e Silva (2019), a automatização contribui para a organização do acervo, a agilidade nas buscas e o acesso às informações. Em bibliotecas que dependem de registros manuais, atividades como controle de empréstimos, devoluções e reservas podem tornar o gerenciamento das informações menos eficiente.

Diante desse cenário, este trabalho apresenta o LibQueue, um Sistema de Gestão de Biblioteca Escolar desenvolvido em Java com o framework JavaFX, estruturado segundo os princípios da orientação a objetos e o padrão arquitetural Modelo-Visão-Controlador (*Model-View-Controller* - *MVC*). O sistema permite o cadastro e gerenciamento de livros, usuários, reservas, empréstimos e devoluções, buscando proporcionar maior organização e praticidade às atividades da biblioteca. Os dados são persistidos em arquivos de texto, adotados como mecanismo de armazenamento da aplicação.

Por ter sido concebido para o ambiente escolar, o sistema implementa mecanismos de controle para o cadastro de usuários por meio de identificadores previamente registrados, diferenciando os perfis de aluno, professor e bibliotecário. No protótipo, esses identificadores simulam registros institucionais, condicionando o cadastro à existência de um identificador válido.

Outro aspecto relevante é o gerenciamento das filas de reserva, que estabelece prioridade para professores em relação a alunos e considera a ordem de solicitação entre usuários de um mesmo perfil. Essa regra busca contemplar situações em que docentes necessitam de exemplares para atividades acadêmicas e desenvolvimento de materiais didáticos.

O artigo está organizado da seguinte forma: a Seção 2 apresenta uma análise comparativa entre o LibQueue e outros trabalhos relacionados; a Seção 3 aborda a organização e implementação do LibQueue; a Seção 4 apresenta as principais funcionalidades e interfaces do sistema; e por fim, a Seção 5 sintetiza os resultados e aponta possibilidades para trabalhos futuros.

2. Trabalhos Similares

Para compreender as diferenças do LibQueue em relação a outras soluções, foram analisados dois sistemas de código aberto: Koha e OpenLibrary. Na Tabela 1 há comparativo resumo das principais funcionalidades dos três sistemas.

O Koha foi desenvolvido em 1999 pela Horowhenua Library Trust, na Nova Zelândia, sendo considerado o primeiro sistema integrado de gerenciamento de bibliotecas (SIGB) de código aberto [Koha Community 2017]. Segundo o manual oficial (IBICT, 2017), o sistema reúne funcionalidades relacionadas ao gerenciamento de usuários, catálogo e circulação. Em comparação, o LibQueue possui uma proposta mais específica para o contexto escolar. Enquanto o Koha permite configurar regras de circulação e reservas, o LibQueue utiliza uma fila de prioridade que favorece professores em relação a alunos e realiza a verificação de atrasos durante o uso do sistema.

Também foi analisado o Open Library, mantido pelo Internet Archive. Diferentemente do LibQueue, a plataforma tem como principal finalidade a catalogação e consulta de informações bibliográficas em escala global, permitindo a colaboração dos usuários na atualização dos registros. O LibQueue, por sua vez, concentra-se no gerenciamento da rotina de uma biblioteca escolar, incluindo cadastro de usuários, controle de empréstimos e devoluções, gestão do acervo e reservas com prioridade.

Tabela 1. Comparativo de funcionalidades entre LibQueue, Koha e OpenLibrary.

Funcionalidade	LibQueue	Koha	OpenLibrary
Usuários	Aluno, professor e bibliotecário	Categorias e tipos de usuários configuráveis	Usuários cadastrados para consulta e colaboração
Limite e prazo de empréstimos	Aluno: 3 livros/7 dias; Professor: 4 livros/10 dias	Definidos por regras de circulação configuráveis	Definidos conforme as políticas de empréstimo digital

Reservas	Fila de prioridade, com preferência para professores	Fila de reservas sem prioridade por perfil	Listas de espera para algumas obras digitais
Atrasos e bloqueios	Verificação automática; usuários em atraso ficam impedidos de novos empréstimos e reservas	Controle de atrasos, notificações e multas mediante configuração	Não possui controle de devoluções físicas ou multas
Acervo	Cadastro baseado em ISBN, nome, autor e gênero	Catálogo baseada no padrão MARC 21	Catálogo bibliográfico colaborativo com metadados, autores, edições e ISBN

A comparação evidencia que, embora Koha apresente maior abrangência e capacidade de parametrização, o LibQueue prioriza funcionalidades voltadas ao contexto de bibliotecas escolares, destacando-se principalmente pelo gerenciamento de filas de reserva por prioridade e pelo controle automático de atrasos. O OpenLibrary, por outro lado, possui uma finalidade predominantemente bibliográfica e colaborativa, não tendo como foco o gerenciamento da circulação de um acervo escolar.

3. Desenvolvimento do LibQueue

O sistema LibQueue foi implementado em Java [Oracle 2026], utilizando JavaFX [OpenJFX 2026] para a construção da interface gráfica e Maven para o gerenciamento das dependências. A aplicação foi estruturada com base nos princípios do padrão MVC, buscando promover a separação de responsabilidades e a organização do código.

O desenvolvimento foi dividido em componentes com responsabilidades distintas:

- **Dados e persistência:** responsáveis pela modelagem, armazenamento e acesso às informações, tendo o pacote *repository* como principal componente de gerenciamento dos dados.
- **Services e Controllers:** os *services* concentram as regras de negócio, enquanto os *controllers* intermediam a interação entre a interface e essas regras.
- **Views:** correspondem às interfaces gráficas, implementadas em FXML com auxílio do *Scene Builder* [Gluon 2026].
- **Utils:** reúne funcionalidades auxiliares utilizadas por diferentes componentes da aplicação, incluindo recursos relacionados à navegação entre telas, gerenciamento de sessão e outras operações de suporte à execução do sistema.

No modelo adotado, as classes do pacote *models* representam as entidades Usuario, Titulo, Livro, Emprestimo e Reserva. Os arquivos FXML correspondem à

camada de Visão, enquanto *controllers* e *services* compõem a camada Controlador, com responsabilidades distintas.

Assim, o fluxo principal segue *View* → *Controller* → *Service* → *Repository*, podendo alcançar o *PersistenceManager* quando uma operação exige leitura ou escrita em arquivos.

A arquitetura adotada, adequada ao caráter acadêmico e prototipal do projeto, permitiu priorizar a implementação das regras de negócio e das estruturas de dados, mantendo uma solução de persistência simplificada em relação ao uso de um Sistema de Gerenciamento de Banco de Dados (SGBD).

3.1 Organização e Estruturação dos dados

3.1.1 Módulo *BibliotecaRepository*

O módulo *BibliotecaRepository*, localizado no pacote *repository*, atua como núcleo central de acesso aos dados da aplicação, concentrando os repositórios responsáveis por usuários, livros, empréstimos, reservas e títulos.

Sua implementação utiliza o padrão Singleton, garantindo uma única instância durante a execução do sistema e permitindo o compartilhamento das estruturas de dados entre os diferentes componentes da aplicação. Durante a inicialização, a classe também participa do carregamento dos dados persistentes e da reconstrução dos relacionamentos entre as entidades.

3.1.4 Módulo Título e Organização das Reservas

O módulo Título é responsável pelo agrupamento lógico das informações associadas a uma mesma obra. Enquanto os exemplares físicos são armazenados individualmente no acervo, cada objeto Título reúne as referências dos exemplares correspondentes a uma determinada obra, bem como seus empréstimos e sua respectiva fila de reservas.

Essa estrutura foi adotada para simplificar o relacionamento entre uma obra e seus exemplares e, principalmente, para permitir que cada título possua uma fila de reservas independente. Dessa forma, a fila de reservas associada a um Título contém somente os usuários que fizeram o pedido de reserva daquela obra vinculadas àquela obra.

Os objetos Título não são persistidos diretamente nos arquivos. Eles são reconstruídos dinamicamente a partir dos dados do acervo durante a inicialização e a atualização das informações do sistema. Como consequência, operações de inserção ou remoção de exemplares exigem a atualização tanto dos dados persistidos quanto das estruturas de agrupamento mantidas pelos títulos. Em contrapartida, essa organização simplifica as operações de consulta e manipulação de empréstimos e reservas, uma vez que as informações relacionadas a uma determinada obra podem ser acessadas diretamente a partir de seu respectivo objeto Título.

Essa modelagem também evita a necessidade de manter uma estrutura global contendo as filas de reservas de todas as obras. Cada título encapsula sua própria fila de

prioridade, mantendo isoladas as reservas correspondentes e facilitando o gerenciamento da ordem de atendimento.

3.1.2 Mecanismos de Armazenamento

A manipulação dos dados em memória utiliza principalmente a interface List e a implementação ArrayList, pertencentes ao Collections Framework. Para encapsular as operações realizadas sobre essas coleções, foram implementadas classes DAO (*Data Access Object*). Esses componentes concentram as operações de inserção, busca, atualização e remoção, evitando que diferentes partes da aplicação manipulem diretamente as estruturas internas de armazenamento. Essa organização contribui para a separação de responsabilidades e facilita a manutenção dos componentes responsáveis pelo gerenciamento dos dados.

3.1.3 Persistência e Reconstrução dos Dados

A persistência da aplicação é realizada por meio de arquivos de texto, gerenciados pela classe PersistenceManager. Cada arquivo representa um conjunto de informações do sistema, e os registros são armazenados em linhas com seus atributos separados pelo caractere *pipe* “|”.

A estratégia de persistência diferencia entidades simples de objetos que possuem relacionamentos. Usuários e livros podem ser reconstruídos diretamente a partir dos registros armazenados. Já empréstimos e reservas mantêm referências para outras entidades e, por isso, são persistidos utilizando identificadores, evitando a duplicação das informações relacionadas.

Durante a inicialização da aplicação, esses identificadores são utilizados para reconstruir as referências entre os objetos. *BibliotecaRepository* participa desse processo, associando os empréstimos aos respectivos usuários e livros e as reservas aos usuários e títulos correspondentes. Dessa forma, os relacionamentos existentes em memória são restaurados sem a necessidade de duplicar os dados das entidades relacionadas nos arquivos.

Os dados são persistidos na pasta data que fica na raiz do projeto. Dentro dela também há uma pasta chamada *seed*, que armazena arquivos que povoam o sistema para testes e demonstração.

3.2 Regras de Negócio e Interação com a Interface

As camadas controlador e serviços intermedeiam a comunicação entre a interface gráfica e os dados da aplicação, mantendo as regras de negócio desacopladas dos elementos visuais.

O pacote service concentra os principais casos de uso do sistema e é dividido entre os módulos *AuthService*, *UsuarioService* e *BibliotecarioService*. A primeira é responsável pela autenticação e pelo cadastro de usuários, enquanto as demais concentram as operações disponíveis aos usuários comuns e aos bibliotecários, respectivamente, incluindo empréstimos, devoluções, reservas e gerenciamento do acervo.

Os *Controllers* recebem as interações realizadas nas interfaces FXML e encaminham as operações aos serviços correspondentes. Foi adotada uma correspondência entre as telas e seus respectivos controladores, facilitando a organização dos eventos e a manutenção da interface.

Um aspecto relevante da implementação é a geração dinâmica de elementos da interface. Em telas que apresentam o catálogo e o inventário de livros, a tabela de exemplares e os *Cards* de filas de reservas e empréstimos ativos, os controllers constroem os componentes informativos a partir dos dados disponíveis em memória. Essas informações são obtidas principalmente da lista de títulos mantida pelo *BibliotecaRepository*. Durante a construção das interfaces, cada título é mapeado e suas informações são utilizadas para gerar os elementos correspondentes, permitindo que a interface reflita o estado atual dos dados da aplicação.

3.3 Interface Gráfica e Componentes Dinâmicos

A interface gráfica do LibQueue foi desenvolvida utilizando arquivos FXML e a ferramenta *Scene Builder* [Gluon 2026], responsáveis principalmente pela definição da estrutura e disposição visual dos componentes. O processamento das interações é delegado aos respectivos *controllers*, mantendo a separação entre apresentação e lógica da aplicação. A interface também utiliza arquivos CSS para a estilização de componentes, como rótulos e botões.

Nas telas que apresentam conjuntos de dados, como catálogos, filas de reservas e históricos de empréstimos, foram utilizados containers flexíveis, como o *FlowPane*. Durante a execução, os controllers geram dinamicamente os cartões informativos e os inserem nesses *containers* de acordo com os dados disponíveis. Essa abordagem permite que a quantidade e o conteúdo dos elementos exibidos acompanhem o estado atual da aplicação, sem a necessidade de definir previamente cada componente no arquivo FXML. No caso do catálogo e do inventário, por exemplo, os dados são obtidos a partir da lista de títulos mantida pelo *BibliotecaRepository*, sendo cada título mapeado para a construção dos elementos correspondentes na interface.

Para a navegação, foram desenvolvidos dois componentes reutilizáveis: *TopBar*, barra superior que apresenta o nome do usuário, a logo do sistema e a opção de logout, e *BottomBar*, barra inferior destinada à navegação entre as telas. Esses componentes são definidos em arquivos FXML independentes e incorporados às demais interfaces por meio do recurso *fx:include*, evitando a duplicação de seus elementos nos diferentes layouts da aplicação. *TopBar* e *BottomBar* são utilizadas nas telas do sistema, com exceção das interfaces de autenticação e cadastro, que possuem fluxos próprios. Cada componente possui ainda um controller responsável pelo gerenciamento de suas respectivas ações. Como os fluxos de navegação dos usuários comuns, representados por alunos e professores, diferem daqueles destinados ao bibliotecário, foram desenvolvidas versões específicas desses componentes para cada perfil.

4. Funcionamento e Interfaces do Sistema

O fluxo de utilização do LibQueue inicia-se pela autenticação do usuário. A partir das credenciais informadas, o sistema identifica o perfil de acesso e direciona o usuário para

a interface correspondente. O sistema possui dois fluxos principais: o destinado a alunos e professores, voltado à consulta, ao empréstimo e à reserva de obras, e o destinado ao bibliotecário, responsável pelo gerenciamento administrativo do acervo e das operações da biblioteca. As principais interfaces são apresentadas nas Figuras 1, 2 e 3, que agrupam, respectivamente, as telas de autenticação, de usuários e de bibliotecários. Os dados apresentados nas figuras correspondem à massa de dados de demonstração utilizada nos testes do sistema



Figura 1. Telas de autenticação e cadastro: (a) login; (b) cadastro de usuário.

Nos perfis de aluno e professor, o catálogo apresenta os títulos disponíveis no acervo e permite a consulta de suas informações. Quando há exemplares disponíveis, o usuário pode solicitar um empréstimo; caso contrário, pode ingressar na fila de reservas associada à obra. As telas de empréstimos e reservas permitem acompanhar os registros ativos e a posição do usuário nas respectivas filas. Nesse processo, o protótipo considera o perfil dos usuários e a ordem de realização das reservas para determinar a ordem de atendimento.

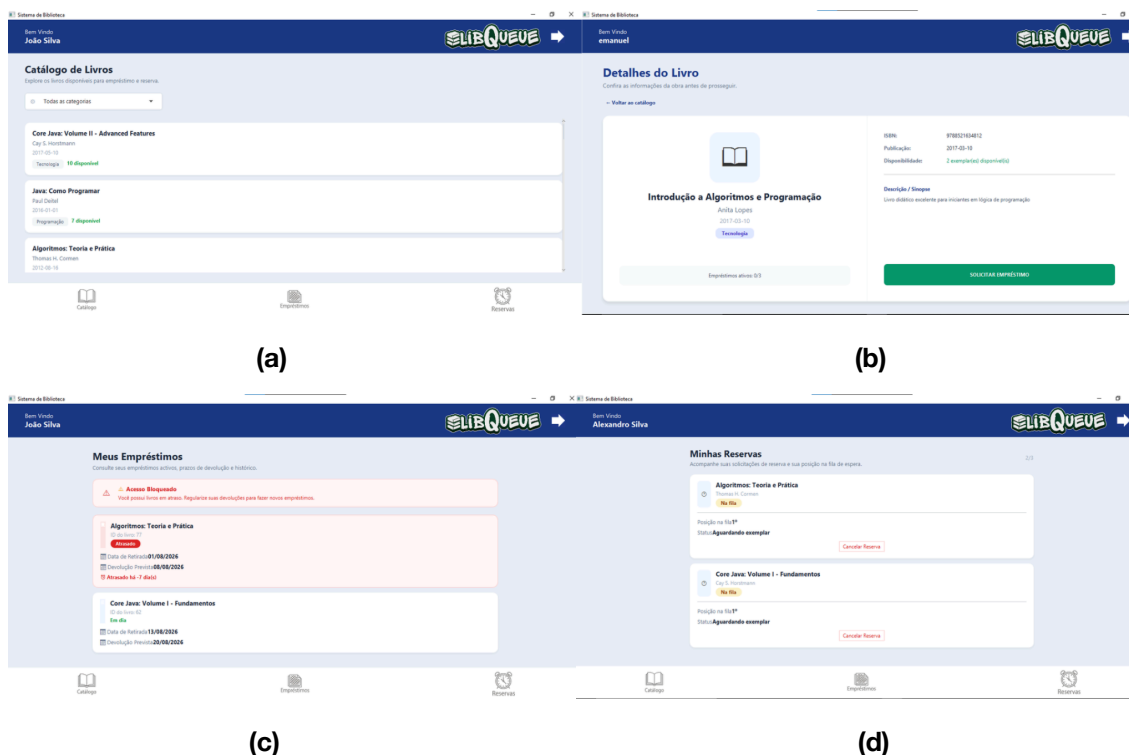


Figura 2. Interfaces dos usuários aluno e professor: (a) catálogo de livros; (b) detalhes de obra disponível; (d) empréstimos ativos; (f) reservas realizadas.

Em relação à realização dos empréstimos, o usuário pode realizar até três empréstimos simultâneos, com prazo de devolução de até sete dias, enquanto o usuário professor pode realizar até quatro empréstimos, com prazo de até dez dias. A diferenciação dos limites e prazos considera as necessidades específicas do perfil docente, incluindo a utilização das obras para a produção de materiais didáticos e a realização de pesquisas. Caso o usuário possua algum empréstimo em atraso ou tenha atingido o limite máximo de empréstimos, o sistema impede a realização de novos empréstimos e reservas até que a situação seja regularizada.

Para o bibliotecário, o sistema disponibiliza interfaces administrativas para o acompanhamento do acervo, o controle de empréstimos e devoluções, o gerenciamento das filas de reserva e o cadastro ou a remoção de exemplares. O Dashboard apresenta indicadores gerais, como o total de livros, os empréstimos ativos e os empréstimos em atraso, enquanto as telas de inventário, fila de reservas e devoluções permitem a execução das operações correspondentes. A partir do card de uma obra no inventário, o bibliotecário pode acessar seus detalhes, consultar a disponibilidade dos exemplares e remover aqueles que não fazem mais parte do acervo ou que foram cadastrados indevidamente.

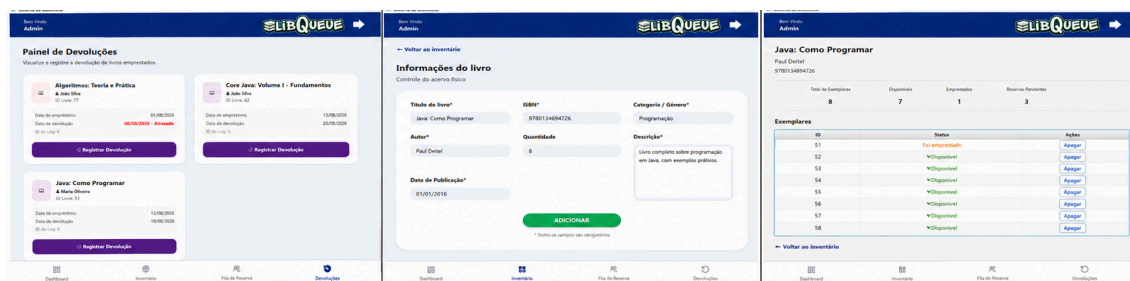
O cadastro de novos exemplares utiliza o ISBN (International Standard Book Number) como identificador da obra. Caso ainda não exista um Título correspondente ao ISBN informado, uma nova instância é criada no BibliotecaRepository. Caso contrário, o exemplar é associado ao Título existente, utilizando os dados bibliográficos já cadastrados para aquele ISBN. Dessa forma, evita-se a duplicação de informações referentes a uma mesma obra.



(a)

(b)

(c)



(d)

(e)

(f)

Figura 3 – Telas do perfil bibliotecário/administrador: (a) Dashboard geral; (b) inventário de títulos; (c) fila de espera por obra; (d) painel de devoluções; (e) gerenciamento e cadastro de exemplares; (f) gerenciamento de exemplares de uma obra.

O sistema também automatiza o atendimento das reservas após uma devolução. Quando um exemplar devolvido possui usuários aguardando na fila de reservas, o sistema registra automaticamente um novo empréstimo para o primeiro usuário da fila, mantendo o exemplar reservado para sua retirada.

4.1 Identificação e Controle de Usuários e Exemplares

Como mecanismo de controle do cadastro de usuários, o sistema utiliza identificadores institucionais fictícios previamente cadastrados. Esses identificadores foram gerados com auxílio de ferramentas de Inteligência Artificial exclusivamente para fins de simulação e testes. O primeiro caractere do identificador permite distinguir o perfil do usuário, utilizando-se *s* para alunos, *t* para professores e *l* para bibliotecários.

Durante o cadastro, o identificador informado é validado pelo sistema, restringindo o registro a usuários que possuam uma identificação previamente autorizada. Essa estratégia foi adotada para simular um mecanismo de controle de vínculo institucional e impedir o cadastro de pessoas que não possuam uma identificação autorizada pela instituição.

Cada exemplar físico possui também um identificador próprio, gerado pelo sistema, permitindo diferenciá-lo dos demais exemplares de uma mesma obra e vinculá-lo individualmente aos registros de empréstimo. No protótipo, esses identificadores são utilizados para fins de demonstração e controle interno. Em uma implementação destinada a um ambiente real, prevê-se sua substituição ou associação a códigos de barras utilizados na identificação física dos exemplares. De forma semelhante, empréstimos e reservas possuem identificadores próprios, permitindo a diferenciação e o controle individual de cada operação registrada.

4.2 Modos de Reinicialização

Para facilitar testes, demonstrações e operações de manutenção, o sistema disponibiliza ao bibliotecário três formas de reinicialização dos dados:

- Reinicialização parcial: remove os empréstimos e as reservas existentes e restabelece todos os exemplares como disponíveis, preservando os dados de usuários e livros. É adequada para cenários em que seja necessário reiniciar as operações de circulação sem reconstruir o acervo.
- Reinicialização para testes e demonstração: substitui os dados atuais pelos dados predefinidos armazenados na pasta *data/seed*, permitindo restaurar rapidamente um estado conhecido da aplicação para testes e apresentações.
- Reinicialização total: remove os dados armazenados no sistema, exceto o cadastro do usuário atualmente autenticado e os identificadores institucionais previamente autorizados para a realização de novos cadastros.

Esses mecanismos facilitam a validação das funcionalidades e permitem restaurar diferentes estados do sistema sem a necessidade de manipulação manual dos arquivos de persistência.

5. Considerações Finais

O desenvolvimento do sistema LibQueue permitiu preencher lacunas operacionais comumente identificadas em sistemas tradicionais de gerenciamento de acervos, como o Koha, ao introduzir uma solução nativa para o gerenciamento de filas reserva com critérios de prioridade entre os perfis dos usuários professor e aluno. A adoção da arquitetura MVC em conjunto com o framework JavaFX na camada de views, provou-se eficiente ao garantir o desacoplamento do código e viabilizar uma atualização de interface gráfica reativa e síncrona com as manipulações de dados em memória.

As decisões de engenharia de software utilizadas, em especial, a abstração fornecida pelo módulo Título, otimizaram a complexidade computacional de buscas. Sob uma perspectiva acadêmica, o projeto demonstrou a viabilidade prática e a utilização de conceitos de estrutura de dados, como lista para a organização e manipulação dos dados e a fila de prioridade para o ordenamento de reservas, importante para cenários acadêmicos reais de instituições educacionais.

Como melhoria futura do projeto, aponta-se uma migração para SGBD para garantir a persistência e integridade dos dados a longo prazo. Ademais, projeta-se o desenvolvimento de um módulo automatizado de comunicação externa para o envio de notificações e avisos de pendências via APIs de e-mail ou mensagens instantâneas, além de uma integração com APIs globais de busca de metadados, como a plataforma *OpenLibrary*.

Referências

Barbosa, E. T. e Silva, A. O. (2019). Implantação de software de gestão de biblioteca escolar na rede municipal de ensino de Vila Velha - ES. In: Congresso Brasileiro de Biblioteconomia, Documentação e Ciência da Informação, 28., Vitória, ES. Anais [...]. Vitória: FEBAB. Disponível em: <https://portal.febab.org.br/cbbd2019/article/view/2314>. Acesso em: 31 maio 2026.

Gluon. (2026). JavaFX Scene Builder Documentation. Disponível em: <https://gluonhq.com/products/scene-builder/>. Acesso em: 16 jun. 2026.

IBICT. (2017). Guia do usuário do Koha. Disponível em: <https://livroaberto.ibict.br/handle/123456789/1064>. Acesso em: 31 maio 2026.

Open Library. (2026). Open Library. Disponível em: <https://openlibrary.org>. Acesso em: 31 maio 2026.

OpenJFX. (2026). OpenJFX Documentation. Disponível em: <https://openjfx.io/>. Acesso em: 16 jun. 2026.

Oracle. (2026). Java Platform, Standard Edition Documentation. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 16 jun. 2026.